

Assignment 1: AWS & API Programming Warm-Up

ComS/CprE 4590X/5590 2026 Spring

Total: 25 points

Due: **Monday, February 9, 11:59 PM**

Learning Objectives

This assignment is designed as a hands-on warm-up to:

- Become familiar with the AWS Management Console and AWS Free Tier
- Practice programmatic access to AWS services using Python and APIs
- Understand basic cloud resources (S3, EC2) and how they are created and managed via SDKs
- Gain experience with simple user-driven, API-based programming

Pre-requisites

AWS Account

You must have your own AWS account. A Free Tier account is sufficient:

<https://aws.amazon.com/free/>

Python Environment

- Python 3.9 or newer is recommended
- You may use VS Code, PyCharm, or any IDE of your choice

AWS SDK for Python (boto3)

Install the required packages:

```
pip install boto3
```

AWS Credentials Configuration

Configure your credentials using one of the standard AWS methods:

- `aws configure`
- or via `~/.aws/credentials` and `~/.aws/config`

as discussed in lecture

You may assume the grader has valid credentials stored in the above default locations.

Part 1 (5 points): Host a Static Website on Amazon S3

Task

Create an Amazon S3 bucket and configure it to host a static website.

- Create an S3 bucket with a globally unique name
- Enable Static Website Hosting
- Upload an `index.html` file (content is up to you)
- Make the website publicly readable

Do not include personal or sensitive information (e.g., ISU ID, AWS account ID).

Hints

You may follow the following steps:

1. Select the S3 service from the AWS Management Console.
2. Create a bucket; in the **Properties** tab of the bucket, enable “Static website hosting” and input `index.html` as the index document. (Note: the endpoint on the tab is the URL to access your website!)
3. In the **Permissions** tab, edit the “Public access settings” by unchecking all four permissions.
4. Add the following policy as the **Bucket policy**:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::bucket-name/*"
    }
  ]
}
```

Note: Replace `bucket-name` with the actual name of your bucket.

Submission

- Submit the public URL of your S3 static website
- Do not delete the bucket until grading is complete

References

Amazon S3 Static Website Hosting documentation: <https://docs.aws.amazon.com/AmazonS3/latest/dev/WebsiteHosting.html>

Part 2 (10 points): Create an EC2 Instance Using Python and AWS APIs

Task

Write a Python program that uses the AWS SDK (`boto3`) to create an EC2 instance.

When executed, your program should interactively prompt the user for the following parameters, one at a time:

- AWS region (e.g., `us-east-1`)
- EC2 instance type (e.g., `t2.micro`)
- AMI ID
- Security group ID
- Key pair name

After collecting the inputs, the program should:

- Call the EC2 API to launch one instance
- Print the instance ID upon successful creation

Submission

Submit one Python source file (`.py`).

Part 3 (10 points): Create an S3 Bucket and Upload a File Using Python

Task

Write a Python program that uses `boto3` to create an S3 bucket and upload a local file.

When executed, your program should prompt the user for:

- AWS region
- Name of the S3 bucket to create
- Path to a local file to upload

The program should then:

- Create the S3 bucket in the specified region
- Upload the file to the bucket
- Print a confirmation message upon success

Submission

Submit one Python source file (`.py`).

Hints

- For Part 1, make your webpage simple to save your credit. (A simple text-only page typically costs only around 0.01 dollar.)
- For Part 2/3, while you are not in the course of testing and after you are done with testing, delete the instances and files, to save your credit.